

DAY 7

HANGMAN PROJECT

HOW TO BREAK A COMPLEX PROBLEM DOWN INTO A FLOWCHART

Introduction To Hangman Game Logic

- It is important we understand how the game works
- to have a better understanding of the game; we could check out the [wiki page](#), and give it a go [online](#).

Using Flowchart to Represent Game Logic

- Come up with a flowchart to represent the flow of the game
- there's no general right answer to this, so freely work on your own flowchart
- but [this](#) is what we'd be using

Takeaway

- Understanding the hangman game is essential to programming it
- Flowcharts help break down complex problems into manageable logical steps
- the game logic involves checking guessed letters, updating the display, and managing lives
- Creating or referring to a flowchart aids in coding and debugging the program

GAME DEMO

Steps 1 – Picking A Random Word And Checking Answers

- To check the to do(s) on the current file you're working on simply go to view, tool windows, and then todo, then on the console make sure it's on current file
- The goal is to be able to achieve what the solution file without opening it, you can actually run the solution file without opening it by right clicking the solution file and select run
- You can loop through strings the same way you can loop through lists using the for loop in python. It also allows you use the characters in the string, the same way you can use the items in the lists.
- Make sure to use the hints in the Task Description when lost.
- It isn't compulsory for the modules you import to be at the foretop of your code, so far you recall python reads line by line, so you can't use the module you imported above the line where it was imported, but it is proper formatting of your code to import at the foretop, so you will get a warning

Step 2 – Replacing Blanks with Guesses

- In this step the focus is creating the visual representation of the word to be guessed in the hangman game, so we'd use “_” to indicate the number of letters the user needs to guess, the next step is to allow the user to guess a letter and update the display accordingly, the display should be the guessed letters in their correct position while unguessed letters remain as blanks
- Read through the description for a more detailed break down of each TODO
- Use hints only if necessary
- Always run the solution first without opening the solution to get an idea of the task
- You don't need to get it right before working on a problem, you can work on a problem and then edit and format for your desired output
- variables are very useful in both making your code cleaner and making code easier

Step 3 – Checking if the Player has won

- In this step, we enhance our game by adding the ability for the user to continue getting letters repeatedly until they have guessed all the letters in the word
- For this we'd implement a while loop to enable continuous guessing, additionally the for loop that updates the display needs to be modified, so it can keep previously guessed letters and add the newly guessed letters till the word is guessed
- the key challenge is preserving guessed letters so that display accumulates correct guesses rather than resetting each time
- run the solution first to get an idea
- it's good when you struggle when learning, you're at the right stage
- you can use the "in" keyword to provide a condition for a while loop, the "in" keyword is used for checking if something exists in something else
- you can also end a while loop from within the loop, you can change the value of the condition running the loop from within, to render the loop false. As python reads code line by line, it wouldn't break until the value of the condition running the loop is changed and the while loop rechecks the condition.
- I used a counter to generate integers to count the times the loop ran, to use as index within the loop or string
- When creating a while loop, create the ending conditions along with it; in Angela's code she created a condition for when the loop should end within the while's block. She stored the true inside a variable to run the while loop, and changed the contents of the while loop after the condition was satisfied
- the way she solved this was by using an elif condition to compare a list of correct letters created outside the while loop(so it doesn't reset with every loop). And the created elif condition compares the letters at a specific point in the loop to the list of correct letters, but remember elif conditions run only if the if fails, so if the guess equals the letter she appends to correct letters, if it isn't she compares with the list of correct letters and then if it is in the list she concatenates to display and if it isn't she uses the else condition to create a blank. And repeats till all blanks are filled or all letters are guessed
- I solved it by creating a variable which contains a list of the placeholders, and by creating a counter and using it as an index to replace items in specific indexes within the variable with the guess if correct, and after the loop's done i convert the list to a string and store in it another variable i then print as display

Step 4 – Keeping Track of the Player's Lives

- this is the easiest step in the project
- “Name can be undefined” means **there’s a possible execution path where the variable might be used before being set**. It’s a warning to help you prevent runtime errors like `UnboundLocalError`.

Example:

python

Copy Edit

```
def check_number(n):  
    if n > 0:  
        result = "Positive"  
    print(result)
```

PyCharm will warn:

Name 'result' can be undefined

Because if `n <= 0`, the `if n > 0:` block won't run, and `result` will not be assigned — yet you're trying to print it afterward.

- remember to read through the to do's and also run the solution prior

Step 5 – Improving The User Experience

- In this chapter, the main aim is to share the code into modules and import these modules, and then clean and improve the logic and application's usability
- we'd be providing feedback to the user, like telling them when they've already guessed a letter, or when they're guess is incorrect
- We'd also indicate how many lives are left
- always pay attention to the colours on your screen
- You can print asterisks to make setting things standout
- You can import using e.g. `import hangman_words` and then use `hangman_words.word_list`, but if the module has only one thing inside, you don't have to type the whole thing out again, you can simply use `from hangman_words import word_list`, this would validate the use of the variable `word_list` throughout the code without you needing to add `(hangman_words.)`, although it's not adviseable if there are multiple things in a single module as you'd have the module over and over to use different elements in it.
- While using `"from hangman_words import "` you can use a "coma" and import multiple things from the same module e.g. `"from hangman_words import stages, logo"`
- user experience is an important concept for a developer
- `print("Hello".upper())` is a thing, to convert a string to upper or lower case you only need to concern yourself with the string in question, and if stored in variable; `variable.lower()`